COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	13/10/2026	**DATE SUBMITTED:**	13/10/2026
NAME:	DYLAN PAOLO M MAGTAAN Serrano, Gabriel Immanuel R.	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

ACTIVITY NO. 12.1 **Algorithmic Strategies**

1. PROCEDURE OUTPUT

Pre-Lab Tip:

When you are given a problem to solve using dynamic programming, the trick is "Not to over-think, because the solution is always hidden in the problem". Most beginners make this mistake of thinking too much, only because they thought the problem was difficult and only a complex solution would solve it. But in most cases the solution turns out to be simple.

Before you start coding, formulate the problem (not the solution) mathematically on a piece of paper. Because a problem well formulated is a problem half solved.

ILO A: Demonstrate an understanding of algorithmic strategies (Pre-Lab Review)

For the pre-lab review, you must fill out the table below. Using algorithms implemented in the previous activities, identify which algorithm satisfies the given algorithmic strategy. For the analysis, you must explicitly provide what part of the algorithm you have identified matches the corresponding strategy.

Strategy	Algorithm	Analysis
Recursion	Binary Search	The algorithm takes the array, low, high, and the search key as parameters. It calculates the middle index using (low+high)/2 and compares the middle element with the search key. If the middle element matches the key, the algorithm returns its index. If the key is smaller than the middle element, it recursively searches the left half by updating high

		to mid-1. If the key is larger, it recursively searches the right half by updating low to mid+1. Each recursive call reduces the search range by approximately half.
Brute Force	Bubble Sort	The bubble sort algorithm repeatedly compares adjacent elements and swaps them if they are in the wrong order. It uses nested loops to check the elements multiple times until the array is sorted. It is considered a brute-force approach because it relies on repeated comparisons and does not use a more advanced strategy to eliminate large portions of the unsorted data.
Backtracking	Depth First Search	The algorithm goes through a path as deeply as possible before backtracking to a previous vertex when there are no more unvisited neighbors. It uses recursion or a stack to keep track of vertices and backtrack when necessary, allowing it to explore other paths in the graph.
Greedy	Selection Sort	The selection sort algorithm divides the array into sorted and unsorted portions. It repeatedly searches the unsorted portion to find the smallest element and swaps it with the first element of the unsorted portion, increasing the sorted portion by one element. It is a greedy approach because it makes the locally optimal choice at each step by selecting the minimum value without considering previously sorted elements. This process continues until the entire array is sorted.
Divide-and-Conquer	Merge Sort	The algorithm repeatedly divides the array into smaller halves until each subarray contains only one element. It then merges the subarrays back together in sorted order by comparing their elements. It follows the divide-and-conquer strategy because it breaks the problem into smaller subproblems, solves them, and combines their results to produce the final sorted array.

This table must be included in your section 6 as table 12-1.

The Problem: MINIMUM STEPS TO ONE

On a positive integer, you can perform any one of the following 3 steps.

1.) Subtract 1 from it. (n = n - 1)
2.) If its divisible by 2, divide by 2. (If n % 2 == 0, then n = n / 2)

3.) If its divisible by 3, divide by 3. (If n % 3 == 0, then n = n / 3).

Now the question is, given a positive integer n, find the minimum number of steps that takes n to 1.

Examples:

1.) For n = 1, output: 0
2.) For n = 4, output: 2 (4 /2 = 2 /2 = 1)
3.) For n = 7, output: 3 (7 -1 = 6 /3 = 2 /2 = 1)

One can think of greedily choosing the step, which makes n as low as possible and continue the same, till it reaches 1. If you observe carefully, the greedy strategy doesn't work here.

Eg: Given n = 10 , Greedy --> 10 /2 = 5 -1 = 4 /2 = 2 /2 = 1 (4 steps).

But the optimal way is --> 10 -1 = 9 /3 = 3 /3 = 1 (3 steps).

So, we need to try out all possible steps we can make for each possible value of n we encounter and choose the minimum of these possibilities.

It all starts with recursion.

$F(n) = 1 + min\{ F(n-1) , F(n/2) , F(n/3) \}\ if\ (n > 1)\ , else\ 0\ (\ i.e.\ , F(1) = 0\ )\ .$

Now that we have our recurrence equation, we can right way start coding the recursion. Wait. Does it have over-lapping subproblems? YES. Is the optimal solution to a given input depends on the optimal solution of its subproblems? Yes... Bingo! it is dynamic programming. So, we just store the solutions to the subproblems we solve and use them later, as in memorization, or we start from bottom and move up till the given n, as in dynamic programming. As it's the very first problem we are looking at here, let's see both the codes:

Memoization

```
int memo[n+1]; // we will initialize the elements to -1 ( -1 means, not
solved it yet ) int getMinSteps ( int n )
{
    if ( n == 1 ) return 0; // base case
    if( memo[n] != -1 ) return memo[n]; // we have solved it already :)
    int r = 1 + getMinSteps( n - 1 ); // '-1' step . 'r' will contain the
    optimal answer finally if( n%2 == 0 ) r = min( r , 1 + getMinSteps( n / 2 )
    ) ; // '/2' step
    if( n%3 == 0 ) r = min( r , 1 + getMinSteps( n / 3 ) ) ; // '/3' step
    memo[n] = r ; // save the result. If you forget this step, then its same as
    plain recursion. return r;
```

```
}
```

Run this code and show the output in table 12-2.

Screenshot	
Analysis	The code uses memoization, which is a dynamic programming technique that stores previously calculated results to avoid repeating the same computations. The getMinSteps() function determines the minimum number of operations needed to reduce a given number to 1 by subtracting 1 or dividing by 2 or 3 when possible. The main driver utilizes a nested for loop before processing each input value,it initializes the memoization array entries to -1, which indicates that the results have not been calculated yet. Then it finally displays the minimum steps required for each input value on the array of 1,4,7, and 10.

Table 12-2. Memoization Implementation

Bottom-Up DP

```
int getMinSteps ( int n )
{
    int dp[n+1], i;
    dp[1] = 0; //
    trivial case for(
    i = 2 ; i < = n ;
    i ++ )
    {
        dp[i] = 1 + dp[i-1];
        if(i%2==0) dp[i] = min( dp[i] , 1+ dp[i/2] );
        if(i%3==0) dp[i] = min( dp[i] , 1+ dp[i/3] );
    }
    return dp[n];
}
```

Run this code and show the output in table 12-3.

Screenshot

```
#include <iostream>
#include <algorithm>

int getMinStepsBottomUp(int n, int memo[]){
    int dp[n+1], i;
    dp[1] = 0;
    for( i = 2 ; i <= n ; i ++ ){
        dp[i] = 1 + dp[i-1];
        if(i%2==0) dp[i] = std::min(dp[i], 1+ dp[i/2]);
        if(i%3==0) dp[i] = std::min(dp[i], 1+ dp[i/3]);
    }
    return dp[n];
}
```

```
// BOTTOM UP
std::cout << "\nBottom Up Dp\n";
for (int t : memoArr){
    std::cout << "n = " << t << "  ->  minimum steps = " << getMinStepsBottomUp(t, memo) << '\n';
}
```

```
int memo[1001];
```

```
int memoArr[] = {1, 4, 7, 10};
```

```
Bottom Up Dp
n = 1  ->  minimum steps = 0
n = 4  ->  minimum steps = 2
n = 7  ->  minimum steps = 3
n = 10  ->  minimum steps = 3
```

Analysis	This code uses a bottom up programming approach to calculate the minimum number of steps needed to reach the given number, starting from 1. It initializes dp[1] = 0 and calculates the minimum steps for each number by subtracting 1 or dividing by 2 or 3 if the number is divisible by those values. It uses std::min to compare the possible steps and stores the smallest value in the dp array. Then on the main driver, we declared an array with the value of 1,4,7,10 which displays the minimum steps needed for each value while avoiding doing repeated calculations by using the previously stored results.

Table 12-3. Bottom-Up Dynamic Programming Implementation

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Create C++ code to implement various algorithmic strategies & ILO C: Create C++ code to solve problems using learned algorithmic strategies

Problem Title: Count the number of paths in a matrix with a given cost to reach the destination cell

Description:
Given an M × N integer matrix where each cell has a non-negative cost associated with it, count the number of paths to reach the last cell (M-1, N-1) of the matrix from its first cell (0, 0) such that the path has given cost. We can only move one unit right or one unit down from any cell, i.e., from cell (i, j), we can move to (i, j+1) or (i+1, j).

For example, consider the following 4 × 4 matrix where cost is 25. Two paths are having a cost of 25. A sample path has been given when the cost is 25.

4	7	1	6
6	7	3	9
3	8	1	2
7	1	7	3

The idea is to use recursion. The problem has optimal substructure. That means the problem can be broken down into smaller, simple "subproblems", which can further be divided into yet simpler, smaller subproblems until the solution becomes trivial.

Deliverables:
- Pseudocode (+ Solving Problem by Hand)
- Working C++ Code

- Analysis of working code
- Screenshots of demonstration

Pseudocode (+Solving Problem by Hand)

```
countPathsWithCost(matrix, row, col, remainingCost):
    if the initial position is out of bounds:
        return 0

    Each function call:
        subtract the cost of the current cell from remainingCost

    If remainingCost is negative:
        return 0

    If we have reached the last cell:
        if remainingCost is 0:
            return 1
        else:
            return 0

    return countPathsWithCost(matrix, row, col + 1, remainingCost)
+ countPathsWithCost(matrix, row + 1, col, remainingCost)
```

Working C++ Code

```cpp
template <int M, int N>
int countPathsWithCost(int (&cost)[M][N], int i, int j, int remaining){
    // Error Check: If the current position is out of bounds, return 0
    if (i >= M || j >= N)
        return 0;

    // Subtract the cost of the current cell from the remaining cost
    remaining -= cost[i][j];

    // If the remaining cost is negative, return 0 as this path cannot reach the last cell
    if (remaining < 0)
        return 0;

    // If we have reached the last cell, check if the remaining cost is exactly 0
    if (i == M - 1 && j == N - 1) {
    //return 1 if the remaining cost is exactly 0, otherwise return 0
        return (remaining == 0) ? 1 : 0;
    }

    // Recursively count the paths by moving right and down
    return countPathsWithCost(cost, i, j+1, remaining) + countPathsWithCost(cost, i+1, j, remaining);
}
```

```cpp
// SUPPLEMENTARY: Count Paths with Cost
int matrix[4][4] = {
    {4, 7, 1, 6},
    {6, 7, 3, 9},
    {3, 8, 1, 2},
    {7, 1, 7, 3}
};

int targetCost = 21;
std::cout << "\nNumber of valid paths: " << countPathsWithCost(matrix, 0, 0, targetCost) << '\n';
```

Analysis of the Working Code (Magtaan)

The function takes the matrix by reference, the current row and column, and the remaining target path cost. First, it checks whether the current position is outside the matrix. If it is, the function returns 0 because that path is invalid. Next, it subtracts the current cell's cost from the remaining target cost. If the remaining cost becomes negative, the function also returns 0 because all cell costs are nonnegative, so the path can no longer reach the target cost. If the function reaches the bottom-right cell, it returns 1 if the remaining cost is exactly 0. This means that it has reached the bottom-right cell with the target cost. Otherwise, it returns 0. If the destination has not been reached, the function recursively explores both the right and down adjacent cells.

The time complexity is O(2^(M+N)) in the worst case, where M is the number of rows and N is the number of columns. Since each call can branch into two more calls

for both right and down direction, the number of calls grows exponentially in the worst case.

Screenshots of Demonstrations (Magtaan)

Test Case 1: 25 Target Cost

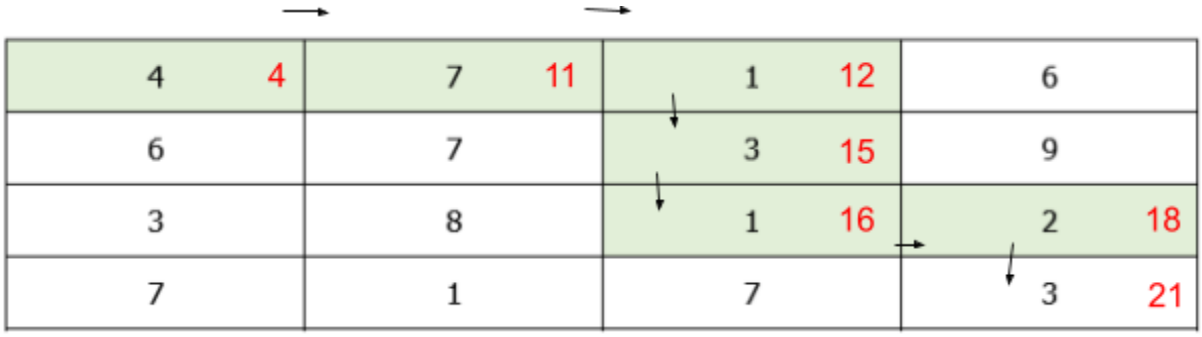
```
Number of valid paths: 0
```

There are no valid paths for a target cost of 25.

Test Case 2: 21 Target Cost

```
Number of valid paths: 1
```

There is one available path for a target cost of 21.



Analysis of the Working Code (Serrano)

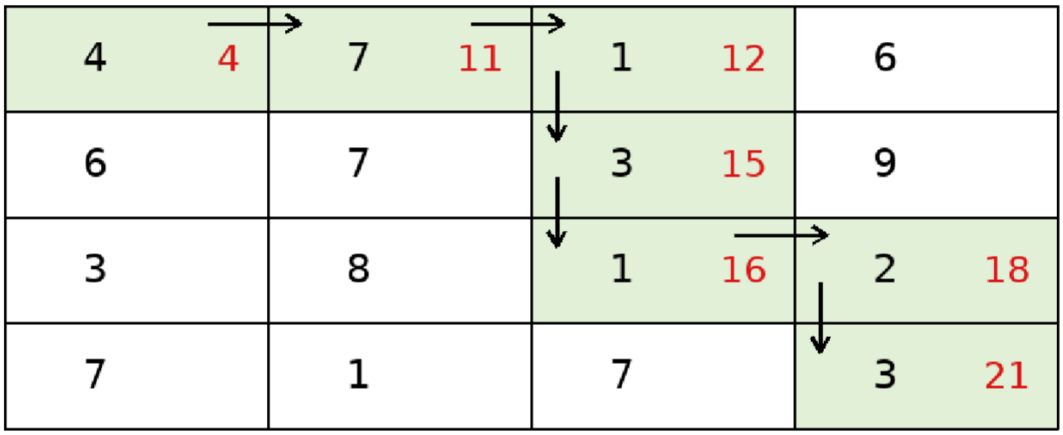
The code uses recursion to count the number of possible paths from the starting cell to the last cell in a 2D array while ensuring that the total cost of each path is equal to the given cost. It moves either right or down and subtracts the cost of each visited cell from the remaining cost. If the position goes out of bounds or the remaining cost becomes negative, the function returns 0 because the path is invalid. Once it reaches the last cell, it returns 1 if the remaining cost is exactly 0, otherwise it returns 0. Finally, it adds the results of the right and down movements to get the total number of valid paths. The time complexity of this code is O(2^(M+N)) because each recursive call branches into two ways, one going to the right and one going down, and it can exponentially grow depending on the dimension of the array.

Screenshot of Demonstrations (Serrano)

Test Case 1 : 21 Target Cost

```
Number of valid paths: 1
```

There is one available valid path for the target cost of 21



Step 1 : (0,0) = 4
Step 2 : (0,1) = 11
Step 3 : (0,2) = 12
Step 4: (1,2) = 15
Step 5: (2,2) = 16
Step 6 : (2,3) = 18
Step 7 : (3,3) = 21

The path that it takes was Right, Right, Down, Down, Right, Down
Since we reached the last step without any remaining left it will return 1.

Test Case 2 : 25 Target Cost

```
Number of valid paths: 0
```

Since our target case in case 2 has no valid path it returned the value 0, If you look closely at the given matrix we can see that it took the path of 4 + 7 + 1 + 6 + 9 = 27 , this results in a negative value of -6.

3. CONCLUSION

Magtaan

Summary of lessons learned
I learned that different algorithmic strategies can be used to solve problems more efficiently depending on their requirements. I also learned how dynamic programming uses memoization and a bottom-up approach to store previous results and avoid repeated calculations. In addition, I learned how recursion can solve problems by breaking them into smaller subproblems, such as counting paths in a matrix with a given cost.
Analysis of the procedure
The procedure helped me understand how recursion, brute force, backtracking, greedy algorithms, and divide-and-conquer work through examples of common algorithms. Implementing memoization and bottom-up dynamic programming also helped me understand how to calculate the minimum number of steps needed to reach while avoiding unnecessary repeated computations.
Analysis of the supplementary
The supplementary activity helped me apply recursion to count the number of paths in a matrix that have a specific total cost. The function checks whether a position is valid, subtracts the current cell's cost, and explores the possible paths to the right and down. This activity helped me understand how base cases and recursive calls work together to solve a problem.
How well did you think you did in this activity?
I think I did well in this activity because I was able to understand the main concepts of algorithmic strategies and apply recursion to the supplementary problem. Although some parts needed careful analysis, I was able to understand how the algorithms work and how they can be used to solve different problems.
What are your areas for improvement?
I need to improve my ability to analyze algorithms, especially in determining their time and space complexity. I also want to practice writing recursive functions, and understanding when to use dynamic programming instead of a basic recursive approach.

Serrano

Summary of lessons learned
I learned how different algorithmic strategies like recursion, brute force, backtracking, greedy, divide and conquer, and dynamic programming solve problems in their own way. I also learned how dynamic programming uses memoization and bottom up approaches and how they store previous results to avoid calculating the same results. I also learned that greedy strategy does not always give the best answer but we just hope that it will give the best answer.
Analysis of the procedure
The procedure helped understand how each of these algorithms really works. For the problem Minimum steps to one problem, it showed the greedy approach but it fails when it comes to having n = 10 because it takes 4 steps instead of 3. Then I learned how memoization and bottom-up dynamic programming work, and they both got the correct answer, since memoization goes top-down using the saved results while bottom-up starts from the smallest case and builds up.
Analysis of the supplementary
For the supplementary activity, we used recursion to count the paths in a matrix that adds up to a given cost. The code works by subtracting the cost of each cell from the remaining cost and moves right or down. It stops when the remaining cost goes below zero, and only counts a path if it reaches the last cell with exactly zero left. This showed me how a base case and recursive solution works together.
How well did you think you did in this activity?
I think I did well in this activity because I was able to understand how each strategy works and apply recursion to solve the supplementary problem. Our code gave the same results as my hand solution so the logic is somewhat correct.
What are your areas for improvement?
I need to understand more about when to use dynamic programming instead of just recursion. Even though our supplementary code and the procedure code both work, I need more practice applying recursive calls in our future codes. I also need to improve my ability in analyzing algorithms and my coding skills.

COMPUTER ENGINEERING

QUEZON CITY

4. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.